

RobustGaSP: MLE vs MAP estimation

37D real data

AOD vs AOD

1 Variable 1: AOD at longitude -171.5625 and latitude 51.875

We start with AOD at the gridbox highlighted in green in Figure 1.

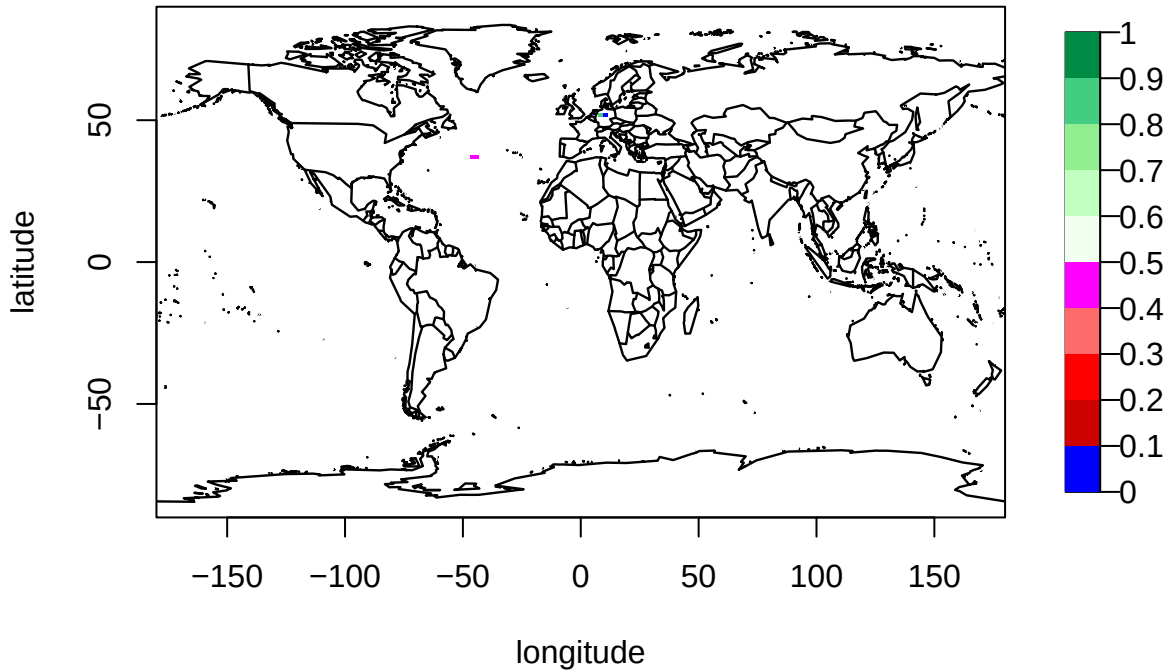
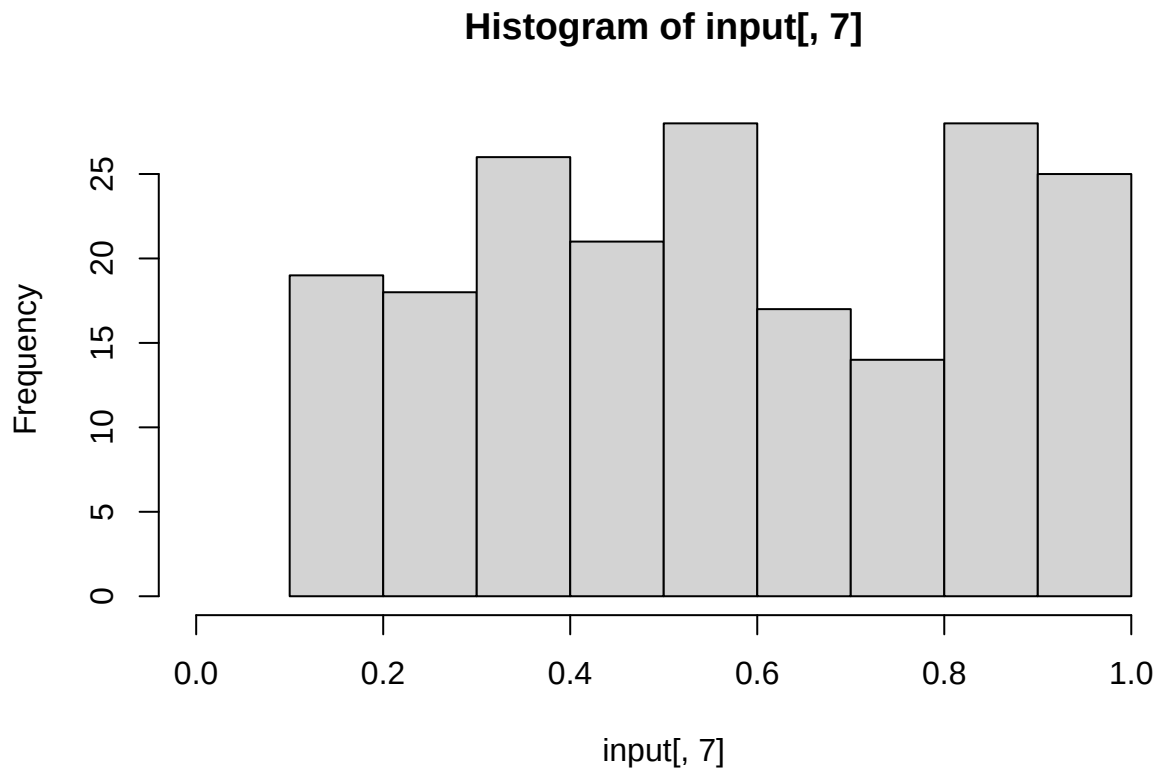


Figure 1: Selected gridbox is highlighted in green

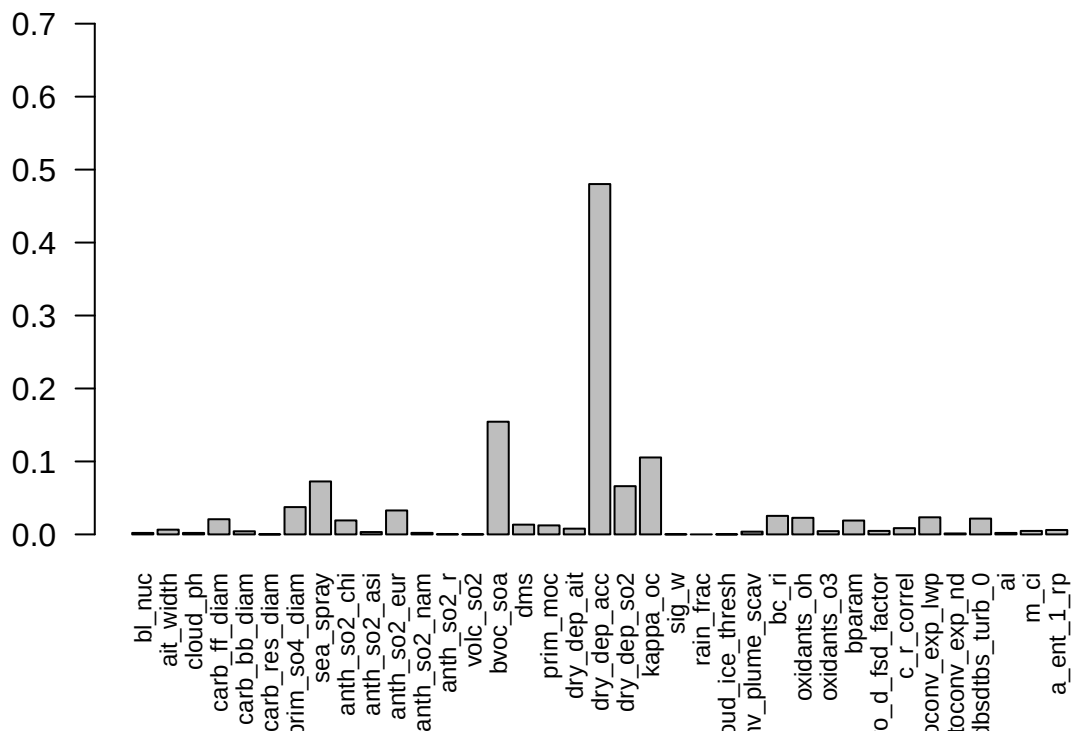
1.1 Emulator fit and validation

SA gets done.



```
## [1] 196
```

```
## [1] 0.1000659894198179244995 0.6579623324796557426453
```



```
## [1] "prim_so4_diam" "sea_spray" "anth_so2_eur" "bvoc_so2"
```

```
## [5] "dry_dep_acc"    "dry_dep_so2"    "kappa_oc"

## [1] 0.03742873995045448848318 0.07260289121546278923169
## [3] 0.03283386992653711949908 0.15445552354786976079204
## [5] 0.48019292643608013637646 0.06611903364363602708664
## [7] 0.10545385769432988332106

## [1] 0.9490868424143702464235
```

```
# input <- input[,which(mainEffects >= th)]
```

```
ncol(input)
```

```
## [1] 37
```

We have a 37-dimensional input, one output (AOD in July at the gridbox with longitude -171.5625 and latitude 51.875), and 196 noise-free observations. We will assume a linear mean function of the form

$$m(x) = \beta_0 + \beta_1(\text{bl_nuc}) x_1 + \beta_2(\text{ait_width}) + \dots + \beta_{37}(\text{a_ent_1_rp})$$

The Gauss (squared exponential) *and* Matern 5/2 kernels will be compared for both estimation methods.

100 new LH points are generated and predictions at these points made. The mean width of the 95% CI's for these predictions are shown in Table 1.

Table 1: Mean width of 95% CI

Estimate method	Kernel	Mean width of 95% CI
MLE	Gauss	0.1378
MLE	Matern 5/2	0.1378
MAP	Gauss	0.1705
MAP	Matern 5/2	0.1705

Leave-one-out validation is carried out; the results are shown in Figure 2.

Going forward, for now, only the Gauss kernel is dealt with; Matern 5/2 will be returned to once I've figured out the estimates returned by the two packages and the derivative of the Matern 5/2 kernel.

1.2 Verifying the estimates by-hand for both estimation methods

To calculate the posterior predictions at the 100 testing points by-hand, we require the matrix H containing the inputs for the 196 observations and well as matrix A , its inverse, and $\mathbf{t}(x^*)$ for all 196 of the new inputs x^* . A and $\mathbf{t}(x^*)$ are built using the estimate of the range parameter returned by each package. By-hand predictions match those returned by the two estimation methods (Figure 3).

Note that for `solve`, which is a numerical way to calculate the inverse of a matrix, if some of the numbers used in the inverse calculation for the matrix A are very small, it assumes that they are zero, leading to the assumption that it is a singular matrix. This is why it specifies that they are computationally singular, because the matrix might not be singular given a different tolerance. The solution is to set the tolerance for when it assumes that they are zero. `solve` allows you to specify this parameter (`tol`).

```
## [1] 0
```

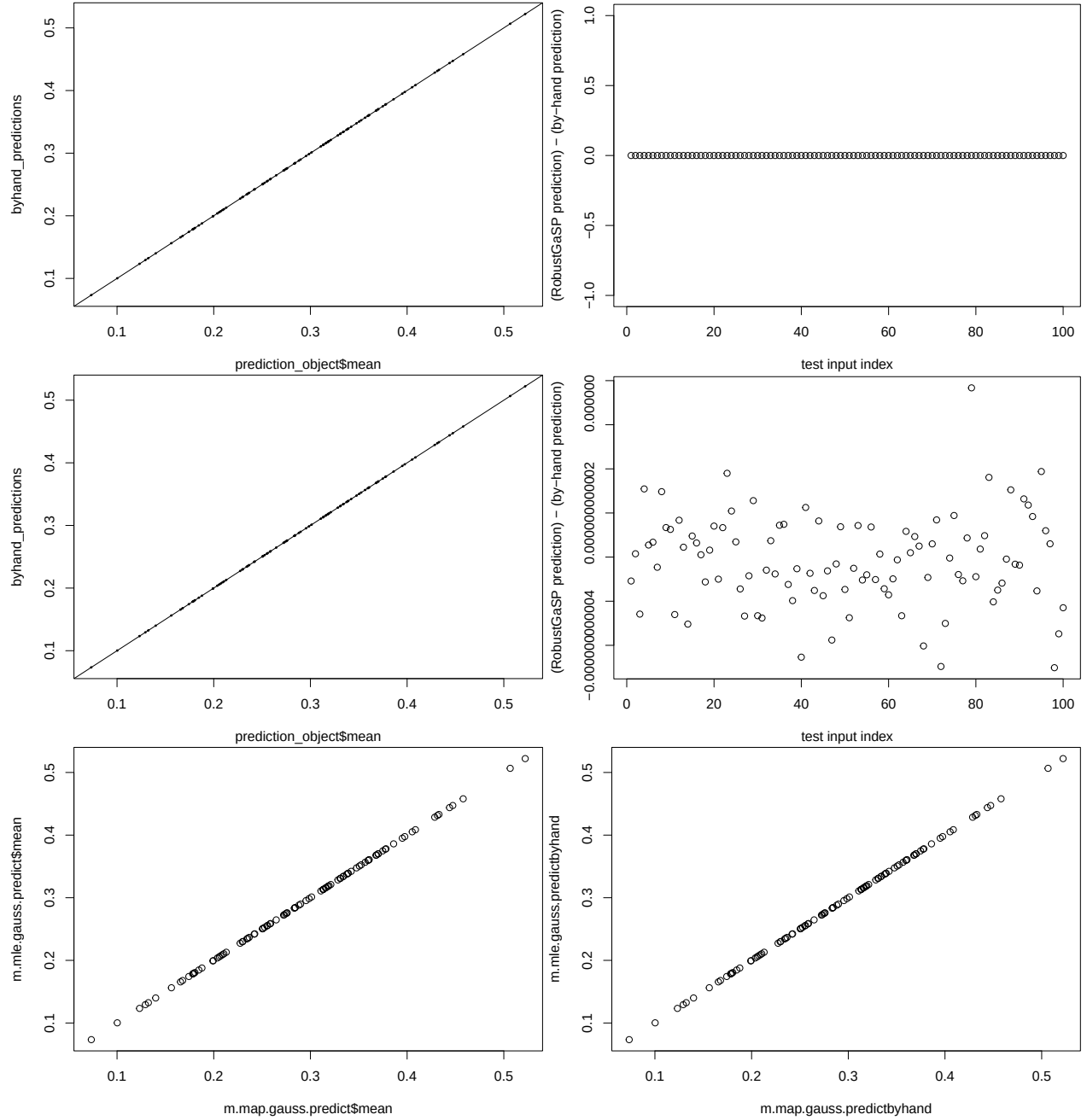


Figure 2: The predictions found by-hand match well with those made by both estimation methods (left), with the latter minus the former found to be essentially zero (right). Top: MLE. Bottom: MAP.

```
## [1] 0.0001625523604609213634831
```

For both we calculate

$$\frac{\sum_{i=1}^N \left| (\text{predictions from package})_i - (\text{predictions by-hand})_i \right|}{N},$$

where N is the number of test points, then divide this by 10^{-8} (machine precision) to get 0 for MLE and 0.0002 for MAP.

We create an object containing the emulated partial derivatives for use in the alignment measure. Finally, we create an object containing the normalised partial derivatives for use in the alignment measure.

2 Variable 2: AOD at longitude -169.6875 and latitude 51.875

We start with AOD at the gridbox highlighted in green in Figure 1.

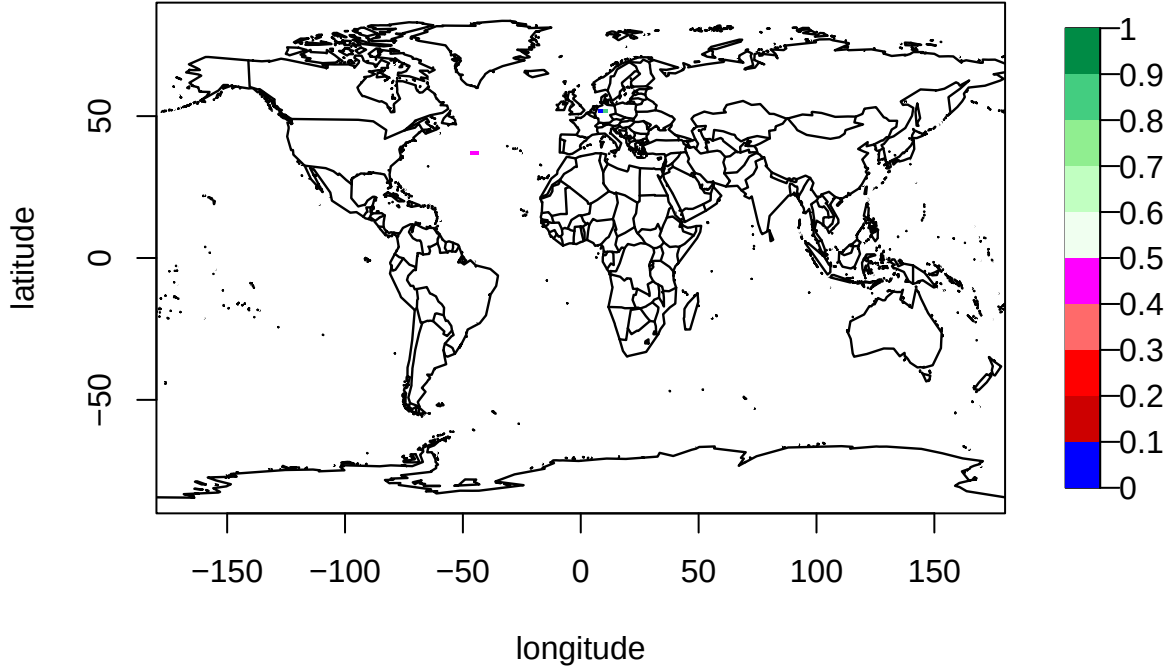


Figure 3: Selected gridbox is highlighted in green

2.1 Emulator fit and validation

```
## [1] 0.1004659438040107488632 0.6631060149520635604858
```

We have a 37-dimensional input, one output (AOD in July at the gridbox with longitude -169.6875 and latitude 51.875), and 196 noise-free observations. We will assume a linear mean function of the form

$$m(x) = \beta_0 + \beta_1(\text{bl_nuc}) x_1 + \beta_2(\text{ait_width}) + \cdots + \beta_{37}(\text{a_ent_1_rp})$$

The Gauss (squared exponential) *and* Matern 5/2 kernels will be compared for both estimation methods.

100 new LH points are generated and predictions at these points made. The mean width of the 95% CI's for these predictions are shown in Table 1.

Table 2: Mean width of 95% CI

Estimate method	Kernel	Mean width of 95% CI
MLE	Gauss	0.138
MLE	Matern 5/2	0.138
MAP	Gauss	0.1708
MAP	Matern 5/2	0.1708

Leave-one-out validation is carried out; the results are shown in Figure 2.

Going forward, for now, only the Gauss kernel is dealt with; Matern 5/2 will be returned to once I've figured out the estimates returned by the two packages and the derivative of the Matern 5/2 kernel.

2.2 Verifying the estimates by-hand for both estimation methods

To calculate the posterior predictions at the 100 testing points by-hand, we require the matrix H containing the inputs for the 196 observations and well as matrix A , its inverse, and $\mathbf{t}(x^*)$ for all 196 of the new inputs x^* . A and $\mathbf{t}(x^*)$ are built using the estimate of the range parameter returned by each package. By-hand predictions match those returned by the two estimation methods (Figure 3).

Note that for `solve`, which is a numerical way to calculate the inverse of a matrix, if some of the numbers used in the inverse calculation for the matrix A are very small, it assumes that they are zero, leading to the assumption that it is a singular matrix. This is why it specifies that they are computationally singular, because the matrix might not be singular given a different tolerance. The solution is to set the tolerance for when it assumes that they are zero. `solve` allows you to specify this parameter (`tol`).

For both we calculate

$$\frac{\sum_{i=1}^N \left| (\text{predictions from package})_i - (\text{predictions by-hand})_i \right|}{N},$$

where N is the number of test points, then divide this by 10^{-8} (machine precision) to get 0 for MLE and 0.0002 for MAP.

We create an object containing the emulated partial derivatives for use in the alignment measure. Finally, we create an object containing the normalised partial derivatives for use in the alignment measure.

3 Alignment measure

```
## [1] 0.9988676808326181211228
```

```
## [1] 0.998867680832555060455
```

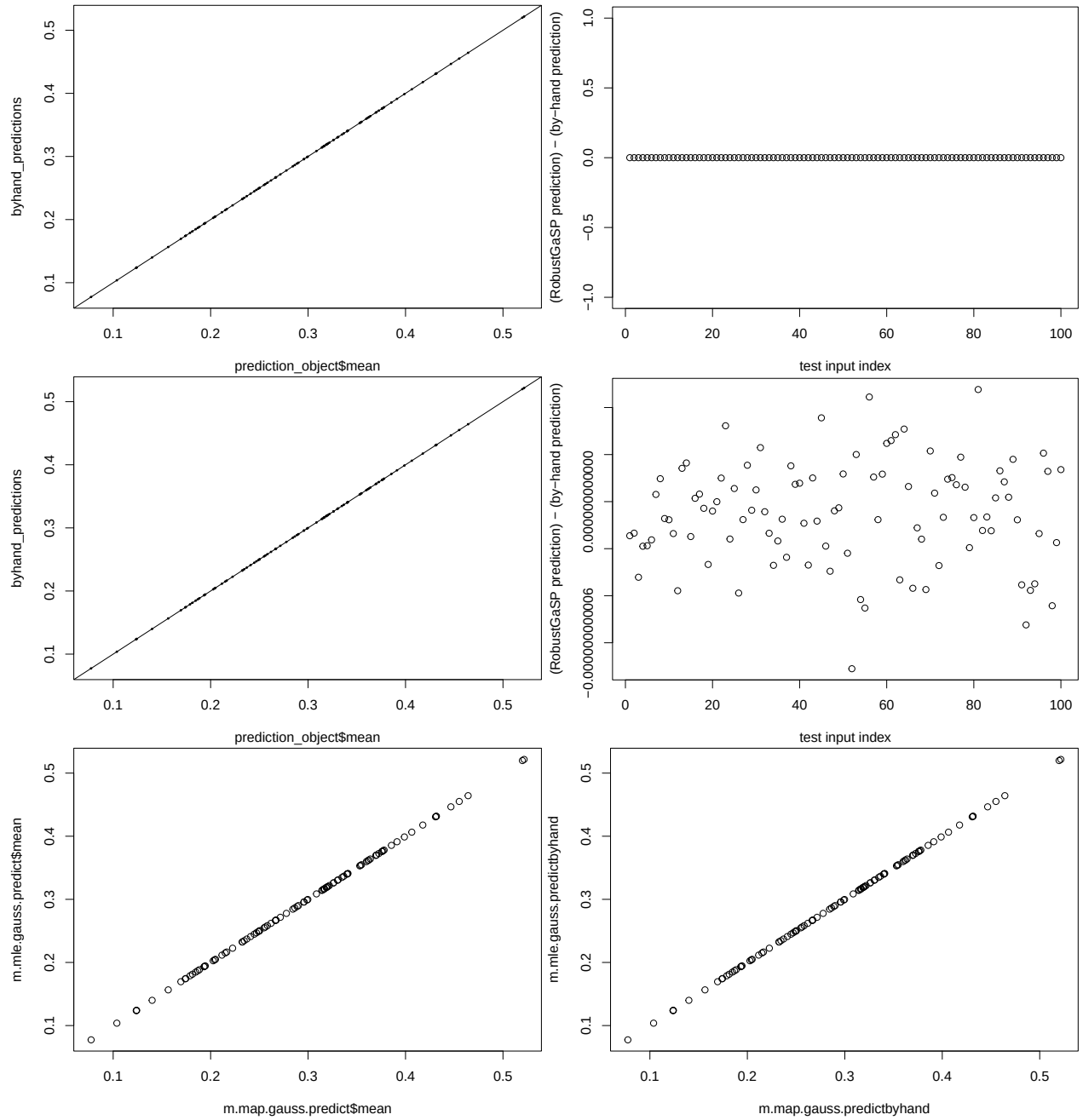


Figure 4: The predictions found by-hand match well with those made by both estimation methods (left), with the latter minus the former found to be essentially zero (right). Top: MLE. Bottom: MAP.